related to `IAsyncDisposable` (introduced in .NET Core 3.0). You can read more about this scenario on my blog at http://mng.bz/zXGB.

With that, you've reached the end of this introduction to DI in ASP.NET Core. Now you know how to register your own services with the DI container, and ideally, you have a good understanding of the three service lifetimes used in .NET. DI appears everywhere in .NET, so it's important to try to get your head around it.

In chapter 10 we'll look at the ASP.NET Core configuration model. You'll see how to load settings from a file at runtime, store sensitive settings safely, and make your application behave differently depending on which machine it's running on. We'll even use a bit of DI; it gets everywhere in ASP.NET Core!

Summary

- When registering your services, you describe three things: the service type, the implementation type, and the lifetime. The service type defines which class or interface will be requested as a dependency. The implementation type is the class the container should create to fulfill the dependency. The lifetime is how long an instance of the service should be used for.
- You can register a service by using generic methods if the class is concrete and all its constructor arguments are registered with the container or have default values.
- You can provide an instance of a service during registration, which will register that instance as a singleton. This approach can be useful when you already have an instance of the service available.